

2024 Sept OPPE 1 - SET 2

Section 1 - Problem 3

Problem Statement

Problem statement:

- You need to write a function **replace_middle_with_n_times_middle**
- that takes a **tuple t of odd length** and **a positive integer n**,
- and **replaces the middle element with n copies of itself**.
- Return a new tuple with the modified content.
- This is a function type question, you don't have to take input or print the output, just have to complete the required function definition.

Example and Explanation:

- Let our input be **t = (1, 2, 3, 4, 5)** and **n = 5**.
- We need to find the middle element at index 2, which is 3, then **replace it with 5 copies of 3**.
- The result should be:
 - **(1, 2, 3, 3, 3, 3, 3, 4, 5)**.
- **Another example:** For t = (1, 2, 3) and n = 4, the middle element is 2 at index 1, so the result should be (1, 2, 2, 2, 2, 3).

Logic Visualization of Index Positions

Tuple with 5 elements: (1, 2, 3, 4, 5)

Indices: 0, 1, 2, 3, 4

Middle index = $5 // 2 = 2$, Middle element = 3

Tuple with 3 elements: (1, 2, 3)

Indices: 0, 1, 2

Middle index = $3 // 2 = 1$, Middle element = 2

Tuple with 7 elements: (10, 20, 30, 40, 50, 60, 70)

Indices: 0, 1, 2, 3, 4, 5, 6

Middle index = $7 // 2 = 3$, Middle element = 40

Pattern Discovery

For odd-length tuple:
middle_index = len(t) // 2

Concept of Tuple Indexing

- In Python, **tuples use zero-based indexing** to access elements by their position.
- **Positive indexing** starts from 0 for the first element and proceeds forward.
- **Negative indexing** starts from -1 for the last element and proceeds backward.
- **The floor division operator //** ensures an integer index when computing positions for odd-length tuples.

Example: **t = (10, 20, 30, 40, 50)**

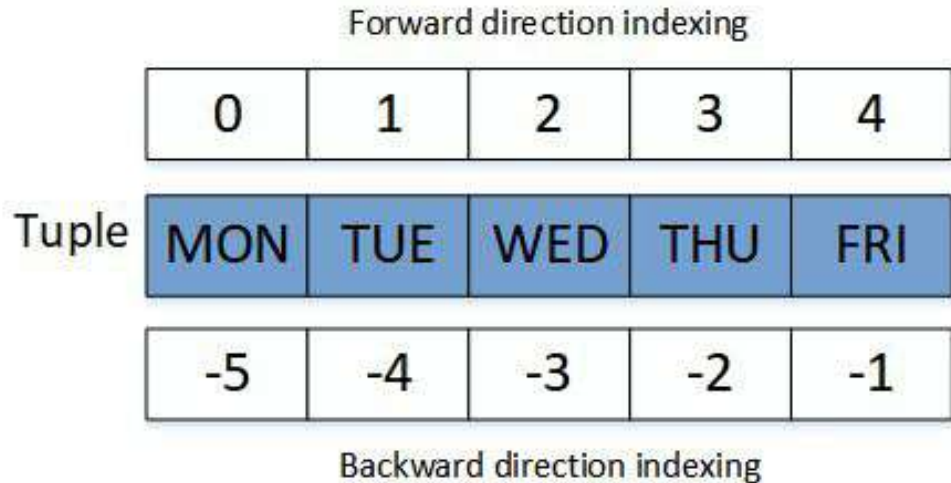
Positive indices: **0, 1, 2, 3, 4**

Negative indices: **-5, -4, -3, -2, -1**

Accessing middle element:

```
middle_index = len(t) // 2
```

```
middle_element = t[middle_index]
```



Concept of Tuple Immutability and Concatenation

- Tuples are **immutable in Python**, meaning their contents cannot be changed after creation.
- You cannot modify an element directly like `t[2] = new_value`.
- To create a modified tuple, you must extract parts, create new elements, and concatenate them using the `+` operator.

Code examples

Tuple concatenation:

Let `t1 = (1, 2)` and `t2 = (3, 4)` be two tuples

`result = t1 + t2` # returns (1, 2, 3, 4)

Multiplying a tuple: `t = (3,) t * 5` # returns (3, 3, 3, 3, 3)

This immutability ensures data integrity **but requires creating new tuples for modifications.**

Logic Flowchart

Step 1: Calculate middle index

```
middle_index = len(t)  
// 2
```

Step 2: Extract middle element

```
middle_element =  
t[middle_index]
```

Step 3: Slice the tuple

```
before_middle =  
t[:middle_index],  
after_middle =  
t[middle_index + 1:]
```

Step 4: Create n copies

```
middle_repeated =  
(middle_element,) * n
```

Step 5: Concatenate all parts

```
result =  
before_middle +  
middle_repeated +  
after_middle
```

Step 6: Return the new tuple

```
return result
```

Our Solution Approach



The Explanatory Approach

Step-by-step implementation with clear variable names for learning and understanding each concept



The Direct Slicing Approach

Using tuple concatenation and inline calculations for balanced readability



The Concise One-Liner Approach

Pythonic compact solution demonstrating advanced tuple manipulation

Each solution produces identical results but demonstrates different Python programming techniques.

Solution 1: The Explanatory Approach

```
def replace_middle_with_n_times_middle(t: tuple, n: int) -> tuple:
    middle_index = len(t) // 2      # Step 1: Find the middle index
    middle_element = t[middle_index] # Step 2: Extract the middle element
    # Step 3: Slice before and after middle
    before_middle = t[: middle_index]
    after_middle = t[middle_index + 1 :]
    # Step 4: Create n copies of middle element
    middle_repeated = (middle_element,) * n
    # Step 5: Concatenate and return
    result = before_middle + middle_repeated + after_middle
    return result
```

A step by step implementation of solution for better understanding

Solution 2: Alternative way Using Direct Slicing

```
def replace_middle_with_n_times_middle(t: tuple, n: int) -> tuple:  
    mid = len(t) // 2  
    return t[:mid] + (t[mid],) * n + t[mid + 1:]
```

How this works:

- Compute the middle index with **mid = len(t)//2**.
- Return everything in one expression: **t[:mid] + (t[mid,])*n + t[mid+1:]**.
- The left slice gets elements before the middle, **(t[mid,])*n** repeats the middle element n times (the comma makes it a tuple), and the right slice gets elements after.
- The + operator concatenates all three parts into the final tuple, avoiding extra variables while staying readable

An alternative solution using one line slicing of tuples

Solution 3: The Concise One-Liner Approach

```
def replace_middle_with_n_times_middle(t: tuple, n: int) -> tuple:  
    return t[:len(t)//2] + (t[len(t)//2],) * n + t[len(t)//2+1:]
```



- This one-liner approach calculates **len(t)//2 inline multiple times** (Python optimizes this),
- performs **slicing, tuple multiplication, and concatenation in one statement** with no intermediate variables needed.
- It demonstrates advanced Python tuple manipulation.
- **Trade-off: Slightly less readable** for beginners but very concise and Pythonic for experienced developers.

The Pythonic Optimization: One-liner solution using single expression solution
Result: Identical output, neater code

Solution comparison

Approach	Pros	Cons	Best For
Explanatory Approach	Highly readable, easy to debug, best for learning, clear variable names	More lines of code, multiple variables	Educational purposes, beginners, complex debugging
Direct Slicing Approach	Balanced readability and conciseness, single return statement, fewer variables	Requires understanding of tuple slicing and multiplication	Production code, intermediate developers, code reviews
One-Liner Approach	Most concise, Pythonic, demonstrates mastery	Harder to debug, less readable for beginners, repeated calculation	Code golf, experienced developers, simple contexts

Key Takeaways

🔑 Tuples use zero-based indexing; middle element of odd-length tuple is at index: `len(t) // 2`

🔑 Floor division operator `//` ensures integer index calculation

🔑 Tuples are immutable - you cannot modify elements in place, must create new tuples

🔑 Tuple concatenation using `+` operator combines multiple tuples into one

🔑 Multiplying a single-element tuple `(element,)` * `n` creates `n` repetitions

🔑 Important: `(element,)` with comma creates a tuple; `(element)` without comma is just a value